```
    chef.add_recipe "mongodb"
    chef.add_recipe "build-essential"
    chef.add_recipe "nodejs::install_from_package"
    chef.json = {
      "nodejs" => {
        "version" => "0.8.0"
        # uncomment the following line to force
        # recent versions (> 0.8.4) to be built from
        # the source code
        # , "from_source" => true
      }
    }
  end
end
```

Let's go through the *config* file to understand what it actually does. First, it tells Vagrant to use the base box named 'precise32' -- the one which we downloaded. Then we configure port forwarding between the Vagrant VM and our host over port 3000, and a shared folder in which we will put our application code. Then we configure some VM hardware settings, such as RAM configuration. Finally, we configure the automation system (*chef_solo*, in this case), and add the recipes from our */cookbooks* directory, defining the applications and services we want to install in the Vagrant VM. Note that I can also pass in specific configurations in JSON format—I'm using this to specify the exact version of Node.js (0.8.0) I want to install.

To power on your custom VM, from inside your application directory issue the following command:

```
$ vagrant up
```

If this is the first time you have issued this command for a new project, Vagrant will actually create the VM from scratch, which may take some time. The next time you enter this command, it will just power the VM on, which is substantially faster. When the 'vagrant up' command is issued, Vagrant works in the background to make the VM required for that environment stand up (as defined in the *Vagrantfile*). If you open up the VirtualBox UI you can actually watch the VM appear, but there's no real reason to do so — you won't need the UI to interact with your VM. Instead, once the VM is up, type the following command:

```
$ vagrant ssh
```

This will automatically give you an SSH session in Vagrant without the need to enter a hostname or credentials. The console output looks similar to what is shown in Figure 1.

Now, to verify the installation of Node, NPM and MangoDB, use the following commands:

```
$ node-v
```

```
mbp15:vagrant-node-mongo constantinecoios$ vagrant ssh
Welcome to Ubuntu 12.04 LTS (GNU/Linux 3.2.0-23-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

121 packages can be updated.
51 updates are security updates.

Welcome to your Vagrant-built virtual machine.
Last login: Fri Sep 14 06:22:31 2012 from 10.0.2.2
vagrant@precise32:~$
```

Figure 1: Vagrant output

```
$ npm -v
$ mongo -version
```

If you use an *ls*, you will also notice a directory */app* located in the Vagrant home directory. Inside will be the application code — these files are actually located on the host system, but are visible on the Vagrant VM as a VirtualBox shared directory. This is where you will run the code. The shared directory means you can pull up Atom in the host OS, and edit the files that will be run in the Vagrant VM. You could start the test application now to see it running, but to get an optimal workflow, you may want something that will restart your application when changes are made to the code. To get this functionality in Node.js, go ahead and install a supervisor on your Vagrant VM from the SSH command line:

```
$ sudo npm install supervisor -g
```

Now, to launch the application, use the command shown below:

```
$ supervisor app/app.js
```

Open up a text editor or IDE in the host system, and open the *vagrant-node/app/app.js* file for editing. Now here's where Vagrant plays a cool part. Open a browser in the host system, and navigate to *http://localhost:3000*. Since the *node.js* app is running in your Vagrant VM, the result will be what's shown in Figure 2.

The application is running in the Vagrant VM, but one can test it in the host browser, as if it was running locally! Now, go to your text editor and modify the *app.js* code to read as follows:

```
var http = require('http');

server = http.createServer(function(req, res) {
  res.writeHead(200, {"Content-Type": "text/html"});
  res.write("<html><body><h1>Hello, from Vagrant!</h1></
body></html>");
  res.end();
}).listen(3000);
```

Refresh the browser and you can see the changes made in the running application.